Implementing optical flow :

Problem statement:

To track an object in a video through sparse optical flow using the Lucas Kanade algorithm.

Approach:

Lucas kanade method states that during a change in a frame the intensity of the pixel of a small part of the frame doesn't change much with time or mathematically:

$$I(x, y, t) = I(x + \delta x, y + \delta y, t + \delta t)$$

Brightness Constancy Rule for optical flow

By taylor expansion we get,

$$I_x u + I_y v + I_t = 0$$

Where,

Spatial Derivative	Optical flow	Temporal derivative
$I_x = \frac{\partial I}{\partial x}$	$u = \frac{dx}{dt}$	$I_t = \frac{\partial I}{\partial t}$
$I_y = \frac{\partial I}{\partial y}$	$v = \frac{dy}{dt}$	

The above equation represents the statement of the Lucas kanade method of sparse optical flow.

Assumptions of the lucas kanade method of sparse optical flow:

- Flow is smooth locally. It means that the motion of objects or points in a small, localized region of an image is relatively consistent and does not exhibit sudden or abrupt changes.
- Neighboring pixels have the same displacement. In other words, neighboring pixels in the selected region are assumed to have similar motion, allowing the algorithm to represent the overall motion with a single vector.

Now let us take a example considering all the assumptions and the statement of lucas:

Let take a 4×4 pixel square is the entire frame so the square gives in total 16 equations which can be represented as below;
(We are considering all pixels in the square have the same optical flow(Assumption 2)):

$$\left[\{I_x(p1) \quad I_y(p1)\} \quad \{I_x(p2) \quad I_y(p2)\}..... \quad \{I_x(p16) \quad I_y(p16)\}\right]^T [u \quad v]^T$$

$$= -\left[I_t(p1) \quad I_t(p2)..... \quad I_t(p16)\right]^T$$

Solving the equation we get,

$x = (M^T M)^{-1} M^T b$ where,

M = $\left[\{I_x(p1) \quad I_y(p1)\} \quad \{I_x(p2) \quad I_y(p2)\}..... \quad \{I_x(p16) \quad I_y(p16)\}\right]^T$

b = $-\left[I_t(p1) \quad I_t(p2)..... \quad I_t(p16)\right]^T$

x = $[u \quad v]^T$

Thank to god that luckily to surpass all this complex mathematics there is a function in python opencv that enables us to do all this mathematics: `cv2.calcOpticalFlowPyrLK()`

It requires the parameters,

prevImg – `first 8-bit input image`

nextImg – `second input image`

prevPts – `vector of 2D points for which the flow needs to be found.`

winSize – `size of the search window at each pyramid level.`

maxLevel – `0-based maximal pyramid level number; if set to 0, pyramids are not used (single level), if set to 1, two levels are used, and so on.`

criteria – `parameter, specifying the termination criteria of the iterative search algorithm.`

The Function returns the following values,

nextPts – `output vector of 2D points (with single-precision floating-point coordinates) containing the calculated new positions of input features in the second image; when OPTFLOW_USE_INITIAL_FLOW flag is`

passed, the vector must have the same size as in the input.

status – output status vector (of unsigned chars); each element of the vector is set to 1 if the flow for the corresponding features has been found, otherwise, it is set to 0.

err – output vector of errors; each element of the vector is set to an error for the corresponding feature, type of the error measure can be set in flags parameter; if the flow wasn't found then the error is not defined (use the status parameter to find such cases).

The code implementation is attached along with the tracked and reference video.

Reference video is 14 to 19 sec of original video timeline which is of 5 sec.

Reference and papers:

- https://docs.opencv.org/3.4/d4/dee/tutorial_optical_flow.html
- https://www.youtube.com/@Computerphile